

ParallelSFL: A Novel Split Federated Learning Framework Tackling Heterogeneity Issues

Yunming Liao^{1,2}, *Yang Xu^{1,2}, *Hongli Xu^{1,2}, Zhiwei Yao^{1,2}
Liusheng Huang^{1,2}, Chunming Qiao³

¹School of Computer Science and Technology, University of Science and Technology of China

²Suzhou Institute for Advanced Research, University of Science and Technology of China

³Department of Computer Science and Engineering, University at Buffalo, the State University of New York
{ymliao98, zhiweiyao}@mail.ustc.edu.cn, {xuyang, xuhongli, lshuang}@ustc.edu.cn, qiao@buffalo.edu

ABSTRACT

Mobile devices contribute more than half of the world's web traffic, providing massive and diverse data for powering various federated learning (FL) applications. In order to avoid the communication bottleneck on the parameter server (PS) and accelerate the training of large-scale models on resource-constraint workers in edge computing (EC) system, we propose a novel split federated learning (SFL) framework, termed *ParallelSFL*. Concretely, we split an entire model into a bottom submodel and a top submodel, and divide participating workers into multiple clusters, each of which collaboratively performs the SFL training procedure and exchanges entire models with the PS. However, considering the statistical and system heterogeneity in edge systems, it is challenging to arrange suitable workers to specific clusters for efficient model training. To address these challenges, we carefully develop an effective clustering strategy by optimizing a utility function related to training efficiency and model accuracy. Specifically, ParallelSFL partitions workers into different clusters under the heterogeneity restrictions, thereby promoting model accuracy as well as training efficiency. Meanwhile, ParallelSFL assigns diverse and appropriate local updating frequencies for each cluster to further address system heterogeneity. Extensive experiments are conducted on a physical platform with 80 NVIDIA Jetson devices, and the experimental results show that ParallelSFL can reduce the traffic consumption by

at least 21%, speed up the model training by at least 1.36×, and improve model accuracy by at least 5% in heterogeneous scenarios, compared to the baselines.

CCS CONCEPTS

- **Human-centered computing** → **Mobile computing**;
- **Computing methodologies** → **Distributed artificial intelligence**.

KEYWORDS

Edge Computing, Split Federated Learning, System Heterogeneity, Statistical Heterogeneity

ACM Reference Format:

Yunming Liao^{1,2}, *Yang Xu^{1,2}, *Hongli Xu^{1,2}, Zhiwei Yao^{1,2}, Liusheng Huang^{1,2}, Chunming Qiao³. 2024. *ParallelSFL: A Novel Split Federated Learning Framework Tackling Heterogeneity Issues*. In *International Conference On Mobile Computing And Networking (ACM MobiCom '24)*, November 18-22, 2024, Washington D.C., DC, USA. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3636534.3690665>

1 INTRODUCTION

With the development of Smart Home, various privacy data (e.g., monitoring data, sensor data) are collected by home devices and stored in the smart gateways without sharing with others [1, 2]. However, these privacy data are extremely valuable and urgently needed for training to further improve the performance of on-device intelligent applications, such as fall detection [3], health monitoring [4, 5], smart fire detection and surveillance [6, 7], intrusion detection [8], and natural language processing [9] for Smart Home. To extract knowledge in the vast amount of privacy data, *edge AI* has been proposed and become a dominant tool [10–12]. One of the popular techniques in edge AI is federated learning (FL), which trains a globally-shared model through collaboration among smart gateways in the data-parallel fashion [13–16].

In FL, each worker (e.g., a smart gateway) is responsible for training an entire model and periodically pushes/pulls the updated/aggregated model to/from the parameter server

* Corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ACM MobiCom '24, Nov. 18-22, 2024, Washington, DC, USA
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0489-5/24/09...\$15.00
<https://doi.org/10.1145/3636534.3690665>

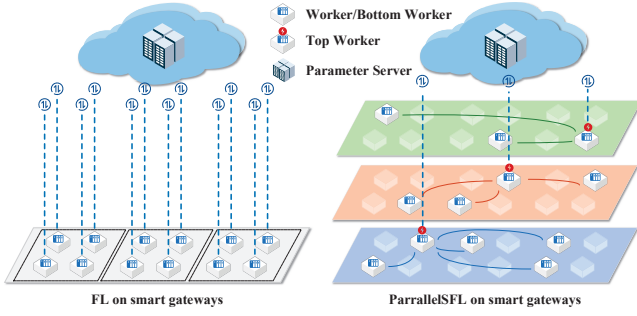


Figure 1: Illustration of FL and ParallelSFL.

(PS) until model convergence, as illustrated in Fig. 1. To boost the performance of AI applications/services, it is necessary and practical to augment the parameters of deep learning models, *e.g.*, large language models (LLMs) with transformer architectures [17–20]. However, due to the hardware limitations of the resource-constrained workers, which usually are only equipped with 1~30TOPS computing power and 1~8GB memory, the requirements of high computing power and large memory for model training hinder each worker from training a complete large-scale model [21–24].

Hence, split federated learning (SFL) has been proposed by incorporating both data parallelism and model parallelism to train large-scale models such as large CNN models and LLMs [22, 25–27]. SFL splits an entire model into two submodels, *i.e.*, bottom submodel and top submodel, at the split layer. The bottom submodel is trained on the resource-constrained workers, while the top submodel is offloaded to the PS. Then the workers perform the training procedure by continuously exchanging the smashed data (also called activations) and gradients with the PS. However, considering the contradiction between the limited bandwidth of PS and numerous workers, the PS may become the system bottleneck, leading to the risk of network congestion and poor scalability [14].

To avoid the communication bottleneck of the PS, we propose a novel SFL framework, termed *ParallelSFL*, which partitions workers into multiple clusters, and encourages each cluster to collaboratively perform the SFL training procedure and exchange entire models with the PS, as illustrated in Fig. 1. Within each cluster, one worker (denoted as the top worker) maintains the top submodel while the remaining workers (denoted as bottom workers) train the bottom submodels. They together perform local updating by exchanging smashed data and gradients. After local updating, the top worker aggregates bottom submodels, and sends bottom and top submodels to the PS for splicing and aggregation. With the benefits of split learning, ParallelSFL can not only release the computing burden on workers for training large-scale models, but also avoid the communication bottleneck and reduce the network traffic of the PS.

However, apart from the resource limitation, ParallelSFL still suffers from two other critical challenges in practical applications. 1) **Statistical Heterogeneity**. Since the local data collected by workers depend on their user preferences and are not shared with others for privacy concerns in Web 3.0, resulting in non-independent and identically distributed (non-IID) data across all workers *i.e.*, statistical heterogeneity [28–32]. The non-IID data slows down the convergence rate and even compromises the accuracy of the trained models [33–35]. 2) **System Heterogeneity**. Workers generally are configured with varying and limited capabilities in EC systems. The computing and communication capabilities of workers could differ from each other by more than tenfold times [36–38]. System heterogeneity poses significant influences on synchronous training processes, as fast workers may be forced to wait for slow ones, leading to increased waiting time and decreased training efficiency [35, 39, 40].

To this end, ParallelSFL is designed to simultaneously tackle the heterogeneity issues through careful and effective worker clustering based on the distinct properties of SFL, which is fundamentally different from the clustering strategy in existing FL works (more details in Sec. 4). On one hand, to address system heterogeneity, it is necessary to shrink the waiting time across workers in each cluster as far as possible, by organizing workers with similar computing/communication capabilities into the same cluster. On the other hand, motivated by the existing works [41], the local data of each cluster together (except the top worker) should be close to IID to deal with statistical heterogeneity. In ParallelSFL, we introduce the KL-divergence to measure the gap between the data distribution of each cluster and IID, and optimize the gap to enhance model accuracy. However, it is intricate to develop the clustering strategy under the heterogeneity restrictions, so as to balance the trade-off between training efficiency and model accuracy. Meanwhile, we assign diverse and appropriate local updating frequencies for clusters to further reduce the waiting time across clusters. In a nutshell, our main contributions are as follows:

- We propose a novel SFL framework, named ParallelSFL, which is designed to overcome the resource limitation and address system as well as statistical heterogeneity by effective cluster partitioning with the distinct properties of SFL.
- We define a utility function with the heterogeneity restrictions to serve as a comprehensive metric for estimating waiting time and data distribution of worker clusters. Upon this, we develop the clustering strategy to balance the trade-off between training efficiency and model accuracy.
- We evaluate the performance of ParallelSFL through a physical platform with totally 80 NVIDIA Jetson edge

devices. The experimental results show that the ParallelSFL can reduce the traffic consumption by at least 21%, speed up the model training by at least 1.36×, and improve model accuracy by at least 5% in heterogeneous scenarios, compared to the baselines.

The rest of the paper is organized as follows. Sec. 2 reviews some related works of SFL and FL. Sec. 3 introduces the background of FL as well as SFL and present our proposed SFL framework. Sec. 4 elaborates the detailed design of our framework. In Sec. 5, we perform extensive experiments to evaluate our framework. Finally, the whole paper is concluded in Sec. 6.

2 RELATED WORK

2.1 Split Federated Learning

The previous vanilla SL methods [42, 43] are proposed to help workers collaboratively train DL models without sharing sensitive raw data in domains such as health care and finance. At that time, the server needs to communicate with workers one by one to complete model training, leading to poor flexibility and scalability. By incorporating FL with SL, SplitFed [25] firstly demonstrates the feasibility and superiority of SFL, and aggregates bottom models after each local updating. Such frequent aggregation results in high network traffic consumption. To save the traffic consumption, LocSplitFed [26] allows the workers not to send features to the PS by using local-loss-based training, which cannot update the top model in time and results in much computing resource. Then, LocFedMix-SL [24] is implemented to maintain all the benefits of SplitFed and LocSplitFed with fixed local updating frequency, but still cannot fully utilize the capacities of heterogeneous workers. In addition, Kim *et al.* [44] and Joshi *et al.* [45] propose SFL approaches without aggregating bottom models to reduce the traffic consumption and accelerate model training, but sacrificing model accuracy. The existing SFL works mainly focus on training large-scale DL models on resource-constrained workers. Although Liao *et al.* [46] present an advanced solution (*i.e.*, AdaSFL), which assigns adaptive and diverse batch sizes for different workers to address system heterogeneity, AdaSFL still cannot deal with the statistical heterogeneity. Despite these notable advancements, none of the existing SFL works have yet explored to simultaneously address system and statistical heterogeneity.

2.2 Federated Learning

Prior to the emergence of SFL, many solutions to address the heterogeneity challenges [35, 37, 38, 47–49] have been studied in typical FL scenarios. In order to alleviate the negative effect of system heterogeneity, Diao *et al.* [50] propose HeteroFL, which enables the training of heterogeneous local

models with varying computation complexities on different workers. Besides, Xu *et al.* [38] investigate to optimize the local updating frequency of different workers, where the workers with high computing/communication capacities are assigned with larger local updating frequencies. To deal with statistical heterogeneity, Sattler *et al.* [51] propose IFCA, the clustered FL method to alternate between estimating the cluster identities and minimizing the loss functions, while Shin *et al.* [52] propose FedBalancer to actively select clients' important training samples. In addition, other works [35, 37, 48] propose to employ worker selection to simultaneously address system and statistical heterogeneity. Specifically, Li *et al.* [35] develop PyramidFL, a fine-grained worker selection strategy that focuses on the divergence between the selected workers and the remaining workers to fully exploit the computing resource and data of different workers. In addition, Luo *et al.* [48] propose AdaSampling and design an adaptive worker sampling algorithm, which tackles both system and statistical heterogeneity to minimize the wall-clock training time. There are also many existing works [53–55] on clustering strategies in FL to deal with system and statistical heterogeneity. However, those FL researches still face the challenge of training large-scale models on resource-constrained workers. Furthermore, those FL researches can not be directly applied for SFL and ParallelSFL, since workers maintaining only the bottom models must exchange smashed data/gradients with the PS (or the top worker) keeping the top model continuously.

3 PRELIMINARY

3.1 Federated Learning

There are N workers and a parameter server (PS) constituting an EC system, where FL is implemented to perform the learning tasks through a decentralized collaboration among the participating workers. In FL, the PS maintains a globally shared model $\mathbf{w}$ with complete structure, while each worker $i \in [N]$ holds its local data $\mathbb{D}_i$ and trains a local model $\mathbf{w}_i$ (*i.e.*, a replica of the global model $\mathbf{w}$). The objective of FL is to find the optimal model $\mathbf{w}^*$ that minimizes the loss function, which can be defined as:

$$\min_{\mathbf{w}} F(\mathbf{w}) \triangleq \frac{1}{N} \sum_{i=1}^N F_i(\mathbf{w}_i) \quad (1)$$

where $F_i(\mathbf{w}_i) = \frac{1}{|\mathbb{D}_i|} \sum_{x \in \mathbb{D}_i} \ell(x; \mathbf{w}_i)$ is the local loss function of worker i , and $\ell(x; \mathbf{w}_i)$ is the loss with respect to the model $\mathbf{w}_i$ and the data sample x .

Due to the inherent complexity of most DL tasks, it is both essential and functional to employ the gradient descent algorithms [56, 57] to solve Eq. (1). In each aggregation round, worker i downloads the global model $\mathbf{w}$ from the PS and independently trains local model $\mathbf{w}_i$ using its own data $\mathbb{D}_i$. The PS

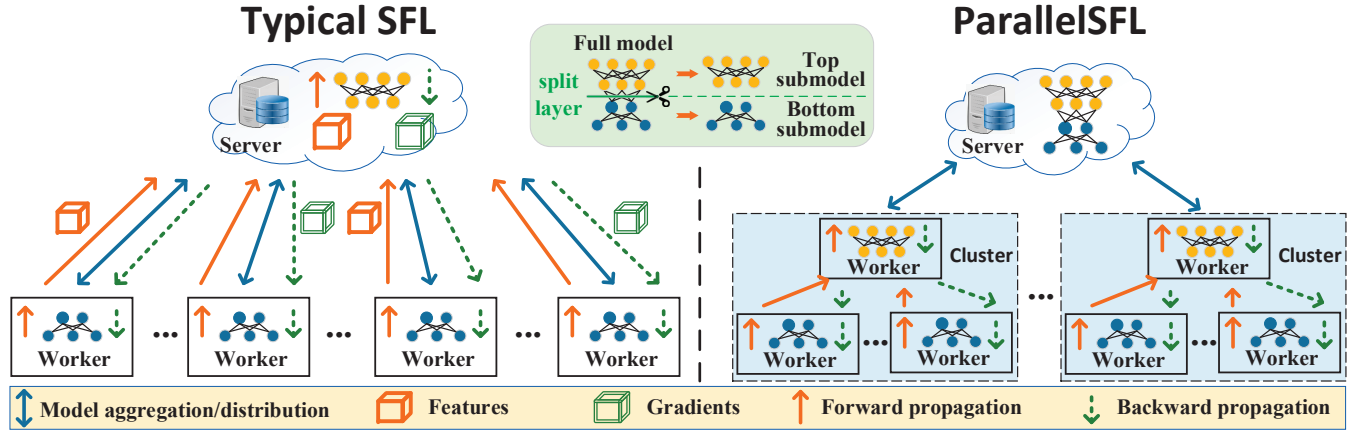


Figure 2: Illustration of typical SFL and ParallelSFL.

aggregates the models from all workers and obtains the updated global model $\mathbf{w} = \frac{1}{N} \sum_{i=1}^N \mathbf{w}_i$ and moves on to the next round. In doing so, the global model can acquire knowledge from the local data of different workers without leaking their data privacy. However, training large-scale models in typical FL remains challenging, as it imposes a significant computation burden on resource-constrained workers and entails substantial communication overhead for model exchange between the PS and workers [24, 26].

3.2 Split Federated Learning

SFL serves as an alternative solution to process complex real-world data using a large-scale model. The fundamental concept of SFL is to split the full model $\mathbf{w}$ into two submodels at the split layer, denoted as $\mathbf{w} = [\mathbf{w}_b, \mathbf{w}_p]$, where $\mathbf{w}_b$ represents the bottom submodel and $\mathbf{w}_p$ represents the top submodel. In the case of a CNN model, the bottom submodel typically comprises the input layer and convolutional layers whereas the top submodel consists of fully-connected layers and the output layer. In SFL, the PS holds the top submodel $\mathbf{w}_p$, while each worker i trains a bottom submodel $\mathbf{w}_{b,i}$ using its local data $\mathbb{D}_i$. By training only the submodels, the computation burden of workers can be significantly reduced, compared to FL.

As illustrated in Fig. 2 (left plot), the basic training process of SFL involves three main stages, *i.e.*, forward/backward propagation of the worker-specific bottom submodels, forward/backward propagation of the top submodel, and global aggregation of bottom submodels on the PS. Firstly, each worker performs forward propagation with a batch of data samples, and delivers the smashed data of the split layer to the PS. Subsequently, the PS conducts forward/backward propagation to update the top submodel. Then, the PS sends the corresponding gradients back to the workers, enabling

them to update their bottom submodels through backward propagation. After local updating, the PS aggregates the bottom submodels from all workers and sends the aggregated bottom submodels back to the workers for further training. However, SFL requires continuous exchange of smashed data/gradients between workers and the PS, which consumes the available bandwidth of the PS and generates a substantial amount of traffic workload. Consequently, the PS may become a system bottleneck, leading to the risk of network congestion and poor scalability.

3.3 Our Proposed SFL Framework

To alleviate the computing/communication burden on the resource-constrained workers and the PS, we propose a novel SFL framework, called *ParallelSFL*, which is designed to tackle heterogeneity issues. Fig. 2 (right plot) illustrates the ParallelSFL framework, where we partition the N workers into C^h clusters in aggregation round h . In the cluster c , there are N_c bottom workers training the bottom submodels and one designated top worker keeping the top submodel. Within each cluster, the bottom workers and the top worker collaborate to perform local updating by exchanging smashed data and gradients. After several local iterations, the top worker aggregates bottom submodels, and sends the bottom submodel and top submodel to the PS for splicing and aggregation. In round h , the bottom submodel on worker i at iteration k is denoted as $\mathbf{w}_{b,i}^{h,k}$, and one iteration for updating the bottom submodel is expressed as:

$$\mathbf{w}_{b,i}^{h,k+1} = \mathbf{w}_{b,i}^{h,k} - \eta \tilde{\nabla} F_{b,i}(\mathbf{w}_{b,i}^{h,k}) \quad (2)$$

Here, $\tilde{\nabla} F_{b,i}(\mathbf{w}_{b,i}^{h,k}) = \frac{1}{|D_i|} \sum_{x \in D_i} \nabla \ell(x; \mathbf{w}_{b,i}^{h,k})$ is the gradient for a certain mini-batch D_i , and $\nabla \ell(x; \mathbf{w}_{b,i}^{h,k})$ denotes the stochastic gradient given the bottom submodel $\mathbf{w}_{b,i}^{h,k}$ and input data

sample x . In round h , let $\mathbf{w}_{p,c}^{h,k}$ represent the top submodel on the top worker of cluster c at the iteration k . Then, the process of one iteration on the top worker is expressed as:

$$\mathbf{w}_{p,c}^{h,k+1} = \mathbf{w}_{p,c}^k - \frac{\eta}{N_c} \sum_{i=1}^{N_c} \tilde{\nabla} F_{p,i}(\mathbf{w}_{p,c}^{h,k}) \quad (3)$$

where $\tilde{\nabla} F_{p,i}(\mathbf{w}_{p,c}^{h,k}) = \frac{1}{|D_i|} \sum_{x \in D_i} \nabla \ell(\mathbf{w}_{b,i}^{h,k}(x); \mathbf{w}_{p,c}^{h,k})$ is the gradient of the top submodel's loss function, and $\nabla \ell(\mathbf{w}_{b,i}^{h,k}(x); \mathbf{w}_{p,c}^{h,k})$ denotes the stochastic gradient for the top submodel $\mathbf{w}_{p,c}^{h,k}$ and the output of the bottom submodel $\mathbf{w}_{b,i}^{h,k}$ (i.e., smashed data), given the input data sample x . After local updating, the top worker aggregates the bottom submodels from all bottom workers in cluster c as follows:

$$\mathbf{w}_{b,c}^h = \frac{1}{N_c} \sum_{i=1}^{N_c} \mathbf{w}_{b,i}^h \quad (4)$$

Subsequently, the top worker in each cluster sends the aggregated bottom submodel and the top submodel to the PS. Finally, the PS splices the submodels into the entire model $\mathbf{w}_c^h = [\mathbf{w}_{b,c}^h, \mathbf{w}_{p,c}^h]$ corresponding to cluster c and aggregates them to obtain the updated global model as follows:

$$\mathbf{w}^{h+1} = \frac{1}{C^h} \sum_{c=1}^{C^h} \mathbf{w}_c^h \quad (5)$$

After that, the PS distributes the updated global model $\mathbf{w}^{h+1}$ to the top worker in new clusters and moves on to the next aggregation round.

To tackle the heterogeneity issues and avoid the communication bottleneck, it is crucial to carefully group the workers into proper clusters to enhance model accuracy and improve training efficiency simultaneously. In Sec. 4.3, we develop the clustering strategy under the heterogeneity restrictions. Although we organize workers with similar computing/communication capabilities into the same cluster, the complete time of one local iteration among clusters still varies significantly considering the heterogeneous workers in diverse clusters. In traditional synchronous schemes, if we assign identical local updating frequencies for all clusters, the fast clusters are forced to wait for the slow ones, incurring idle waiting time and significantly destroying the training efficiency. Generally, the clusters with higher performance can be allocated with larger local updating frequencies, while those with lower performance are allocated with smaller ones. Therefore, the waiting time among clusters would be greatly reduced.

4 SYSTEM DESIGN

4.1 Overview

The overall design of ParallelSFL is illustrated in Fig. 3. ParallelSFL consists of four key modules at the PS side, i.e.,

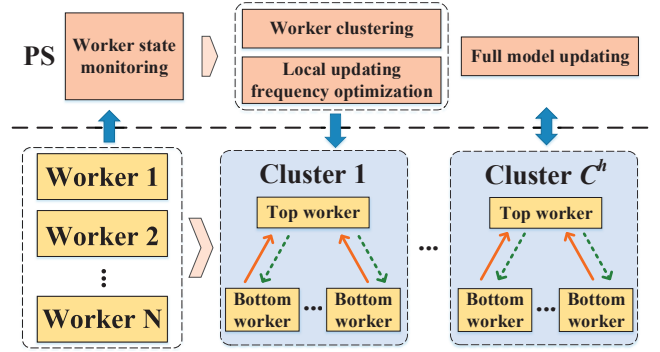


Figure 3: Overview of ParallelSFL.

worker state monitoring, worker clustering, local updating frequency optimization, and full model updating. The worker side mainly consists of the local updating module. At the beginning of each aggregation round, the worker state monitoring module starts to collect the state information (e.g., ingress bandwidth, label distribution, computing and communication capabilities) from all workers. After that, the worker clustering module in the PS organizes the workers into suitable clusters to tackle heterogeneity issues. Subsequently, the local updating frequency optimization module determines diverse and appropriate local updating frequencies for clusters to further reduce the waiting time across clusters. Meanwhile, the full model updating module distributes the full model to the top worker of each cluster, and then the top worker delivers the bottom submodels to the bottom workers in the cluster. Within each cluster, the bottom workers continuously exchange smashed data/gradients with the top worker at each iteration. After a certain number of local iterations, the top worker aggregates the bottom submodels from bottom workers in each cluster. Then, the PS splices and aggregates the top and bottom submodels from the top workers in each cluster, and moves on to the next aggregation round.

4.2 Worker State Monitoring

In order to make an effective clustering strategy, it is necessary to monitor the current states of all workers (e.g., ingress bandwidth, label distribution, computing and communication capabilities).

In EC, each worker i has a limited available ingress bandwidth B_i^h in each round h . The top worker in each cluster typically consumes a significant portion of the bandwidth to exchange smashed data/gradients with bottom workers. To prevent the top worker from becoming a bottleneck, it is instinct to ensure that the occupied bandwidth of the top worker does not exceed the available ingress bandwidth. At the beginning of a certain round, the PS collects and analyzes the statistical distribution of the ingress bandwidth of

all workers. Then the PS employs the statistical results as the available ingress bandwidth for each worker.

Secondly, the local data of each cluster together (except the top worker) should be close to IID. Herein, the label distribution, a vector $\mathbf{V} = \{v_j \geq 0, j \in [1, M]\}$ ($\sum_{j=1}^M v_j = 1$) to parameterize a categorical distribution of class labels over M classes, is required to assist in worker clustering. In typical SFL, the workers deliver the smashed data with corresponding labels to continue forward propagation of the top submodel on the PS, which enables the PS to directly collect the labels of workers' smashed data and generates the label distribution $\mathbf{V}_i$ of the mini-batch of worker i [22, 25, 26].

Thirdly, the collection of time-varying computing and communication capacities of workers is necessary for worker clustering and local updating frequency optimization. On one hand, we denote the computing time of one iteration on the bottom submodel and top submodel for worker i in round h as $\mu_{b,i}^h$ and $\mu_{p,i}^h$, respectively. The relationship between these two parameters is clear and fixed, so that we only monitor $\mu_{b,i}^h$. On the other hand, $\beta_{i,j}^h$ is indicated as the transmission time of the smashed data/gradients at one iteration between worker i and worker j in round h . As proxy metrics, we adopt $\mu_{b,i}^h$ and $\beta_{i,j}^h$, which can be recorded by the workers directly during model training, to indicate the computing and communication capacities of worker i in round h , respectively. Prior to starting model training in round h , the PS collects the latest $\hat{\mu}_{b,i}^h$ and $\hat{\beta}_{i,j}^h$ from all workers. Besides, to improve the robustness of the estimation, we introduce the moving average with the historical states of workers [58]. Accordingly, the PS estimates $\mu_{b,i}^h$ and $\beta_{i,j}^h$ on worker i in aggregation round h by calculating the moving average with $\alpha \in [0, 1]$ (e.g., 0.8 in our experiments) as:

$$\mu_{b,i}^h = \alpha \cdot \mu_{b,i}^{h-1} + (1 - \alpha) \cdot \hat{\mu}_{b,i}^h \quad (6)$$

$$\beta_{i,j}^h = \alpha \cdot \beta_{i,j}^{h-1} + (1 - \alpha) \cdot \hat{\beta}_{i,j}^h \quad (7)$$

Since the size of the information about worker states (e.g., 100-300KB [59]) is much smaller than that of model parameters, it is reasonable to ignore the cost (e.g., bandwidth consumption and time cost) for state monitoring [60].

4.3 Worker Clustering

Based on the collected state information, ParallelSFL partitions the workers into C^h clusters and determines the top worker for each cluster. Besides, each cluster is configured with suitable local updating frequency.

In each cluster c , there are N_c bottom workers and one designated top worker. Due to the constraint of the available ingress bandwidth B_c^h for the top worker, the number of participating workers N_c to simultaneously train the bottom submodels is limited. We denote the bandwidth occupied by

Algorithm 1: Worker clustering in round h

Input: $B_i^h, \mathbf{V}_i, \mu_{b,i}^h, \mu_{p,i}^h, \beta_{i,j}^h, K$.

Output: The worker clustering strategy.

- 1 Calculate $KL(\Phi_i || \Phi_j)$ for all workers i and $j, i \neq j$.
 - 2 Divide the workers into K sets $S_1, \dots, S_K$
($S = S_1 \cup \dots \cup S_K$) by calling the K -means algorithm.
 - 3 Initialize $c = 1$.
 - 4 **while** S is not empty **do**
 - 5 Select the worker i with maximum B_i^h from S_k with the most workers as the top worker for cluster c .
 - 6 Remove worker i from its set.
 - 7 Denote the distribution of cluster c as Φ_c .
 - 8 Select the worker j with the maximum t_j^h from each S_k into set A .
 - 9 Select workers into the cluster c from set A to make $KL(\Phi_0 || \Phi_c)$ smallest under the constraints of Eqs. (8) and (10).
 - 10 $c \leftarrow c + 1$.
 - 11 Minimize the utility function $\sum_{c=1}^{C^h} \mathcal{U}_c$ by exchanging workers from different clusters, without violating Eqs. (8) and (10).
-

each worker at each iteration as b . The occupied bandwidth in each cluster is limited as follows:

$$N_c \cdot b \leq B_c^h \quad (8)$$

The size of cluster c depends on the available ingress bandwidth B_c^h of the top worker. Since the local data of the top worker does not contribute to model training, we should control the number of the top worker (i.e., the number of the clusters) as small as possible.

For the cluster c , we can formulate the completion time t_i^h of one iteration on the worker i in round h as:

$$t_i^h = \mu_{b,i}^h + \beta_{i,c}^h + \mu_{p,c}^h \quad (9)$$

where $\mu_{p,c}^h$ and $\beta_{i,c}^h$ separately denote the computing time of one iteration of the top worker and the transmission time of the smashed data at one iteration from bottom worker i to the top worker in cluster c . Since the top worker has to simultaneously exchange smashed data/gradients with multiple workers, in order to ensure training efficiency, the number of participating workers N_c in cluster c is limited as:

$$N_c \cdot \mu_{p,c}^h \leq \max\{\mu_{b,i}^h + \beta_{i,c}^h\}, \forall i \in [N_c] \quad (10)$$

Besides, the waiting time of the bottom worker i can be defined as $t_{c,o}^h - t_i^h$, where $t_{c,o}^h = \max\{t_i^h\} (\forall i \in [N_c])$ denotes the completion time of one iteration in round h for the slowest worker in cluster c . Accordingly, the average waiting

time of cluster c at one iteration in round h is formulated as:

$$\mathcal{W}_c^h = \frac{1}{N_c} \sum_{i=1}^{N_c} (t_{c,o}^h - t_i^h) \quad (11)$$

To minimize the waiting time of all workers in cluster c , we should group the N_c workers, whose completion time of one iteration is close enough to each other, into the same cluster.

To tackle the non-IID issue, the data distribution of each cluster needs to be close to IID. We first define the IID distribution as Φ_0 . If the data of all workers follows IID distribution, we get $\Phi_0 = \frac{1}{N} \sum_{i=1}^N \mathbf{V}_i$, where $\mathbf{V}_i$ is the label distribution of worker i . In cluster c , the label distribution of N_c workers is:

$$\Phi_c = \frac{1}{N_c} \sum_{i=1}^{N_c} \mathbf{V}_i \quad (12)$$

Then, we introduce the KL-divergence $KL(\Phi_c || \Phi_0)$ to measure the gap between Φ_c and Φ_0 as follows [61, 62]:

$$KL(\Phi_c || \Phi_0) = \sum_{j=1}^M \Phi_c(v_j) \log \frac{\Phi_c(v_j)}{\Phi_0(v_j)} \quad (13)$$

To deal with statistical heterogeneity, it is necessary to control the KL-divergence $KL(\Phi_c || \Phi_0)$ as small as possible.

Taking system and statistical heterogeneity into account, it is challenging to partition these workers into appropriate clusters. We normalize $\mathcal{W}_c^h$ as well as $KL(\Phi_c || \Phi_0)$, and introduce a utility function to evaluate the effect of cluster c in round h as follows:

$$\mathcal{U}_c = \lambda \cdot \mathcal{W}_c^h + (1 - \lambda) \cdot KL(\Phi_c || \Phi_0) \quad (14)$$

where λ is a weight coefficient used to balance $\mathcal{W}_c^h$ and $KL(\Phi_c || \Phi_0)$. In round h , we need to carefully partition all the workers into appropriate clusters to minimize $\sum_{c=1}^{C^h} \mathcal{U}_c$ under the constraints of Eqs. (8) and (10), so that we can simultaneously address system and statistical heterogeneity and implement efficient ParallelSFL.

We propose a greedy algorithm to make the effective clustering strategy. Firstly, by the K -means algorithm (e.g., $K = N/5$), according to the KL-divergence of label distribution among workers, we divide the workers with small KL-divergence into the same set and obtain K sets $S_1, \dots, S_K$ (Line 1-2 of Alg. 1). Next, we greedily select the workers with maximum ingress bandwidth B_c^h from the set S_k with the most workers as the top worker for cluster c . We construct the set A including the worker j with the maximum t_j^h from each set S_k and group workers into the cluster c from set A to make $KL(\Phi_0 || \Phi_c)$ smallest under the constraints of Eqs. (8) and (10). Subsequently, we repeat the above operations to further partition the remaining workers and create new clusters, until all workers are grouped into suitable clusters (Line 4-10 of Alg. 1). Finally, we optimize the distribution of workers among clusters through fine-tuning, aiming to

minimize the utility function $\sum_{c=1}^{C^h} \mathcal{U}_c$, while adhering to the constraints specified in Eqs. (8) and (10) (Line 11 of Alg. 1). Upon completion of this process, we obtain the C^h effective clusters in round h .

4.4 Local Updating Frequency Optimization

Additionally, due to the system heterogeneity, the complete time of one iteration among clusters is highly different. We denote the communication time for the top worker in cluster c transmitting the submodels to the PS in round h as β_c^h . The local updating frequency of cluster c is defined as τ_c^h . Accordingly, the completion time t_c^h of aggregation round h for the cluster c is expressed as:

$$t_c^h = \tau_c^h \cdot t_{c,o}^h + \beta_c^h \quad (15)$$

Besides, the waiting time of cluster c can be defined as $t^h - t_c^h$, where $t^h = \max\{t_c^h\} (\forall c \in [C^h])$ denotes the completion time of aggregation round h for the slowest cluster. Accordingly, the average waiting time of all clusters in round h is formulated as:

$$\mathcal{W}^h = \frac{1}{C^h} \sum_{c=1}^{C^h} (t^h - t_c^h) \quad (16)$$

To minimize the average waiting time, ParallelSFL regulates the local updating frequencies of all clusters so as to align their completion time of one round. Thus, it ensures that the average waiting time will be small enough to mitigate the negative impacts of the synchronization barrier and improve training efficiency. The regulation rule is expressed as:

$$\lfloor \frac{\tau_c^h \cdot t_{c,o}^h + \beta_c^h}{\tau_l^h \cdot t_{l,o}^h + \beta_l^h} \rfloor = 1 \quad (17)$$

where l denotes the index of the fastest clusters assigned with the default maximum local updating frequency τ in round h . According to Eq. (17), we can obtain the specific local updating frequencies for all clusters in round h

4.5 Full Model Updating

After cluster c performs local updating in round h , the top worker firstly aggregates the bottom submodels from other workers according to Eq. (4). Then the top worker splices the bottom submodel and top submodel, and sends the full model to the PS for global aggregation. Considering the clusters with diverse numbers of workers are configured with different local updating frequencies for model training, the full model in each cluster is updated and trained with varying degrees, which needs adaptive aggregation weight to guarantee the performance of aggregated global model [38, 63, 64]. Therefore, the PS aggregates the entire models from different clusters with adaptive weights related to the

local updating frequencies and the size of clusters as follows:

$$\mathbf{w}^h = \sum_{c=1}^{C^h} \frac{N_c \cdot \tau_c^h \cdot \mathbf{w}_c^h}{\sum_{c=1}^{C^h} N_c \cdot \tau_c^h} \quad (18)$$

The aggregated global model is stored in the PS and will be distributed to future clusters to continue further training, or be used for AI tasks.

4.6 Discussion of Privacy Protection

In existing SFL literature [25, 42, 43], there are two ways, *i.e.*, sharing labels and returning logits (without sharing labels), to complete the model updating, which can also be applied in ParallelSFL. Generally, the bottom workers usually deliver the features with corresponding labels to continue forward propagation of the top model on the top worker. However, if the bottom workers are unwilling or not permitted to share labels, the top worker can return the output logits of the top model to the workers for calculating the loss locally. Then the bottom workers send the loss values back to the top worker for deriving the gradients and complete the backward propagation. Moreover, the existing privacy-preserving techniques such as Differential Privacy [65–67] and Homomorphic Encryption [68] can be employed to further protect the privacy of the smashed data in ParallelSFL. In this paper, we focus on efficient model training on resource-constrained workers in case of system and statistical heterogeneities, which is orthogonal to privacy protection.

5 EVALUATION

5.1 Experimental Settings

System Deployment. We conduct extensive experiments to evaluate the performance of ParallelSFL on an edge computing hardware prototype system. Specifically, we employ a deep learning GPU workstation as the PS, which is equipped with an Intel(R) Core(TM) i9-10900X CPU, four NVIDIA GeForce RTX 2080Ti GPUs and 256 GB RAM. In addition, we specify 80 NVIDIA Jetson kits¹, including 30 Jetson TX2 devices, 40 Jetson NX devices, and 10 Jetson AGX devices, as workers to construct a heterogeneous system. The detailed technical specifications of Jetson TX2, NX, and AGX are listed in Table 1. Notably, the TX2 showcases a 256-core Pascal GPU with 6 TOPs and a CPU cluster consisting of a 2-core Denver2 and a 4-core ARM CortexA57. The NX is outfitted with a 384-core NVIDIA Volta GPU with 21 TOPs and a 6-core NVIDIA Carmel ARMv8.2 CPU. Jetson Xavier NX dramatically enhances the NVIDIA software stack over 10× the performance of Jetson TX2. Lastly, the AGX stands out with a 512-core NVIDIA Volta GPU with 32 TOPs and an

Table 1: Device technical specifications.

	AI Performance	GPU Type
Jetson TX2	6 TOPs	256-core Pascal
Jetson NX	21 TOPs	384-core Volta
Jetson AGX	32 TOPs	512-core Volta
	CPU Type	ROM
Jetson TX2	Denver 2 and ARM 4	8 GB LPDDR4
Jetson NX	6-core Carmel ARM 8	8 GB LPDDR4x
Jetson AGX	8-core Carmel ARM 8	32 GB LPDDR4x
	CPU Frequency	GPU Frequency
Jetson TX2	2.0GHz	1.12GHz
Jetson NX	1.9GHz	1.1GHz
Jetson AGX	2.2GHz	1.37GHz

8-core NVIDIA Carmel ARMv8.2 CPU. Besides, the computing power of these devices is at the same level as the current mainstream smart gateways.

In the experiments, we build the software platform based on Docker Swarm [69, 70] and the PyTorch deep learning library [71]. The Docker Swarm, a distributed software development kit, facilitates the construction of a distributed system and enables the monitoring of each device’s operational status. The PyTorch library facilitates the implementation of model training on devices. Additionally, to streamline communication among devices, we implement MPI (Message Passing Interface) [72], which includes a collection of sending and receiving functions.

Settings of System Heterogeneity. To enable the workers with heterogeneous computing and communication capabilities, we present the following experimental settings.

1) **For Computation.** All the Jetson TX2, NX, and AGX devices can be configured to work with different modes, which specifies the number of working CPUs and the frequency of CPU/GPU for the devices to work with different computing capacities. Specifically, TX2 can work in four modes each while NX and AGX work in one of eight modes. Devices working in different modes exhibit diverse capabilities. For instance, the AGX with the highest performance mode (*i.e.*, mode 0 of AGX) achieves training by 100× faster than the TX2 with the lowest performance mode (*i.e.*, mode 1 of TX2). To further reflect the time-varying on-device resource, we randomly change the modes for devices every 20 rounds.

2) **For Communication.** All devices are connected to the PS via WiFi routers. We arrange 80 devices into four groups, each containing 20 devices. These groups are then placed at different locations, *i.e.*, 2m, 8m, 14m, and 20m away from the WiFi routers. Due to random channel noise and competition among devices, the bandwidth between the PS and devices dynamically varies during the training. The bandwidth of

¹<https://docs.nvidia.com/jetson/>

devices is measured by iperf3 [73], which dynamically fluctuates between 1Mb/s and 30Mb/s during training.

Applications and Models. We evaluate the performance of ParallelSFL on four applications (*i.e.*, classical datasets) with four DNN models.

1) **Speech Recognition.** The Google Speech dataset [74] (expressed as Speech for short) is adopted for the task of speech recognition, which allows a computer or device to recognize and interpret spoken language. The dataset includes 85,511 and 4,890 audio clips for training and test, respectively. The model trained on Speech is a plain CNN model with four 1-D convolutional layers and one fully-connected layer ².

2) **Object Recognition.** We adopt the CIFAR-10 dataset [75] for the evaluation, which is an image dataset composed of 60,000 32×32 color images (50,000 for training and 10,000 for test) across 10 categories. We utilize an 8-layer AlexNet with a size of 136MB [76] for CIFAR-10. The AlexNet is composed of three 3×3 convolutional layers, one 7×7 convolutional layer, one 11×11 convolutional layer, two fully-connected hidden layers, and one softmax output layer.

3) **Image Classification.** ImageNet [77] is a dataset for image recognition that consists of 1,281,167 training images, 50,000 validation images, and 100,000 test images from 1000 categories. To adapt to the resource-constrained workers, we create a subset of ImageNet, called IMAGE-100, which contains 100 out of 1,000 categories. We adopt a famous large model VGG16 with a size of 321MB [78] for the complex task, which consists of 13 convolutional layers with the kernel of 3×3, two fully-connected layers, and a softmax output layer.

4) **Natural Language Processing.** We utilize the Question-answering Natural Language Inference (QNLI) dataset [79], a classification dataset consisting of question-sentence pairs, in this application. There are 104,743 samples for training and 5,463 samples for test in the QNLI dataset. We train a RoBERTa model with a size of 501MB [17] on QNLI dataset. The RoBERTa is composed of 12 transformer layers, a 768×2 fully-connected layer, and a softmax output layer.

Settings of Statistical Heterogeneity. In the experiments, training samples of each worker are drawn independently by a vector $\mathbf{v}$. To create non-IID datasets, we draw from a Dirichlet distribution [80, 81], *i.e.*, $\mathbf{v} \sim \text{Dir}(\delta \mathbf{q})$, where $\mathbf{q}$ characterizes a prior class distribution, and $\delta > 0$ is a concentration parameter controlling the identicalness among workers. With $\delta \rightarrow \infty$, all workers have identical distributions to the prior class distribution (*i.e.*, IID); with $\delta \rightarrow 0$, each worker holds data samples from one class, which indicates a high degree of statistical heterogeneity. We specify 6 values (*e.g.*, ∞ , 1, 0.5, 0.25, 0.2, 0.1) for δ to generate different data distributions that cover a spectrum of identicalness, and

define $p = 1/\delta$ (*i.e.*, $p = 0, 1, 2, 4, 5, 10$) to quantify the non-IID levels. The degree of statistical heterogeneity increases as p increases, and $p = 0$ is a special case of IIDness.

Baselines. We measure the effectiveness of ParallelSFL through a comparison with four approaches.

1) **IFCA** [51] is a famous clustered FL approach that alternates between estimating the cluster identities and minimizing the loss functions to deal with statistical heterogeneity.

2) **LocFedMix-SL** [24] is a typical SFL approach, which proposes to reduce the aggregation frequency of bottom submodels to save the traffic consumption, but can not fully utilize the capacities of heterogeneous workers.

3) **AdaSampling** [48] is an advanced FL approach that focuses on designing an adaptive worker sampling algorithm to tackle both system as well as statistical heterogeneity and minimize the wall-clock convergence time.

4) **AdaSFL** [46] is a state-of-the-art SFL approach, which assigns adaptive local updating frequency and diverse batch sizes for heterogeneous workers to enhance the training efficiency without addressing statistical heterogeneity.

Metrics. We adopt the following metrics to evaluate the performance of ParallelSFL and the baselines.

1) **Test Accuracy** reflects the accuracy of the models trained by different approaches on the test datasets, and is measured by the proportion of the data correctly predicted by the models to all the test data. We evaluate the test accuracy of the global model in each round, and record the final accuracy for all approaches.

2) **Training Time** is denoted as the total wall clock time taken for training a model to achieve a target accuracy. For a fair comparison, we set the target accuracy as the achievable accuracy by all approaches. In addition, we also record the average waiting time of all workers to reflect the training efficiency of different approaches.

3) **Network Traffic** is calculated by summing the traffic for transmitting models or features between the PS and workers or between the bottom workers and top workers when achieving a target accuracy.

Experimental Parameters. By default, each set of experiments will run 100 aggregation rounds for RoBERTa on QNLI, and 250 aggregation rounds for CNN on Speech, AlexNet on CIFAR-10, and VGG16 on IMAGE-100. The learning rates and decay rates for CNN, AlexNet, and VGG16 are identical, and are initialized as 0.1 and 0.993 [30, 82], respectively, while for RoBERTa, the learning rate is initialized as 0.001 without decay rate. Besides, the batch size is set as 16 for RoBERTa and 64 for the remaining three models. For the SFL approaches, we separately split the CNN, AlexNet, VGG16, and RoBERTa at the 4th, 5th, 13th, and 3rd layer [46].

²https://pytorch.org/tutorials/intermediate/speech_command_classification_with_torchaudio_tutorial.html

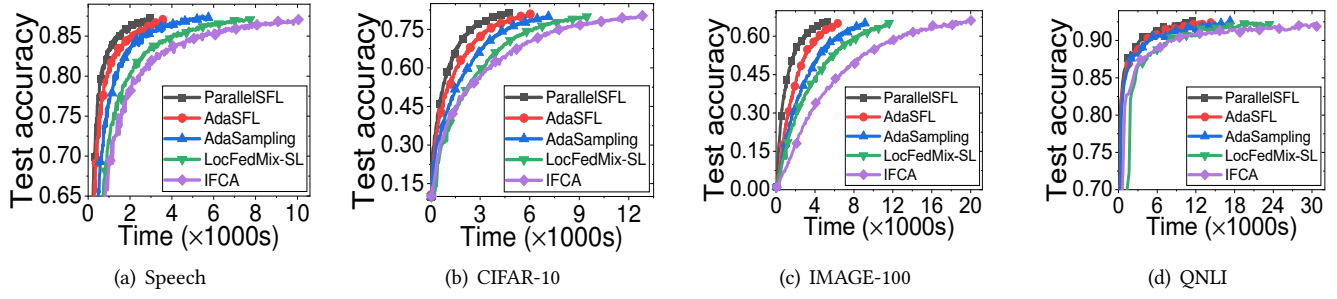


Figure 4: Test accuracy of five approaches on the four IID datasets.

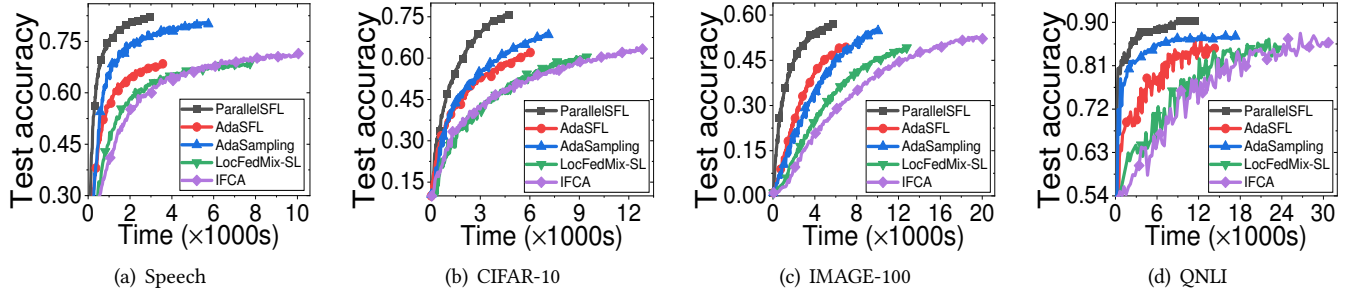


Figure 5: Test accuracy of five approaches on the four non-IID datasets.

5.2 Overall Performance

Firstly, we conduct a set of experiments on the four IID datasets to evaluate the performance of ParallelSFL and the baselines. The training processes of five approaches are presented in Fig. 4. By the results, all the approaches achieve similar test accuracy eventually on the four datasets. However, ParallelSFL achieves the fastest convergence, followed by AdaSFL, which is much faster than the other approaches on all the four datasets. For instance, as illustrated in Fig. 4(a), ParallelSFL takes 2,893s to achieve 87% accuracy for CNN on Speech, while AdaSFL, AdaSampling, LocFedMix-SL, and IFCA consume 3,267s, 4,877s, 6,878s and 9,253s, respectively. Similarly, by Fig. 4(b), for AlexNet on CIFAR-10, ParallelSFL can separately speed up training by about 1.36 $\times$, 1.65 $\times$, 2.18 $\times$ and 2.87 $\times$, compared to AdaSFL, AdaSampling, LocFedMix-SL, and IFCA, respectively. AdaSFL with adaptive and diverse batch sizes for heterogeneous workers speeds up the convergence rate, while ParallelSFL with adaptive worker clustering and local updating frequency optimization overcomes the system heterogeneity and achieves the fastest convergence. Specifically, for VGG16 on IMAGE-100, as shown in Fig. 4(c), ParallelSFL reduces the total training time by about 17%, 43%, 55%, and 68%, compared to the baselines (*i.e.*, AdaSFL, AdaSampling, LocFedMix-SL, and IFCA). Moreover, by Fig. 4(d), ParallelSFL takes 10,838s to achieve

92% accuracy for RoBERTa on QNLI, while AdaSFL, AdaSampling, LocFedMix-SL, and IFCA consume 13,635s, 16,812s, 22,565s, 29,190s, respectively.

Secondly, we also conduct a set of experiments of these approaches on the four datasets with non-IID level $p=10$, and the results are presented in Fig. 5. We observe that all the approaches maintain a similar convergence rate as that in the IID scenario, but suffer from varying degrees of accuracy degradation. However, ParallelSFL with adaptive worker clustering and model splitting achieves the highest accuracy among these approaches. For instance, by Fig. 5(a), ParallelSFL achieves 82.1% accuracy in 2,972s for CNN on Speech, while AdaSFL, AdaSampling, LocFedMix-SL, and IFCA takes 3,567s, 5,747s, 7,737s, and 9,885s to reach the accuracy of 68.64%, 80.07%, 68.23%, and 71.31%, respectively. Similarly, as shown in Fig. 5(b), for AlexNet on CIFAR-10 with the same training time of 4,300s, ParallelSFL improves the test accuracy by about 24.12%, 19.08%, 36.45% and 37.28%, compared to AdaSFL, AdaSampling, LocFedMix-SL and IFCA, respectively. IFCA with alternate worker clustering improves the accuracy to a certain extent, while AdaSampling with adaptive worker sampling speeds up the convergence rate and improves test accuracy. However, ParallelSFL with adaptive worker clustering and model splitting tackles both system and statistical heterogeneity better than IFCA and AdaSampling. Specifically, Fig. 5(c) illustrates that ParallelSFL separately improves

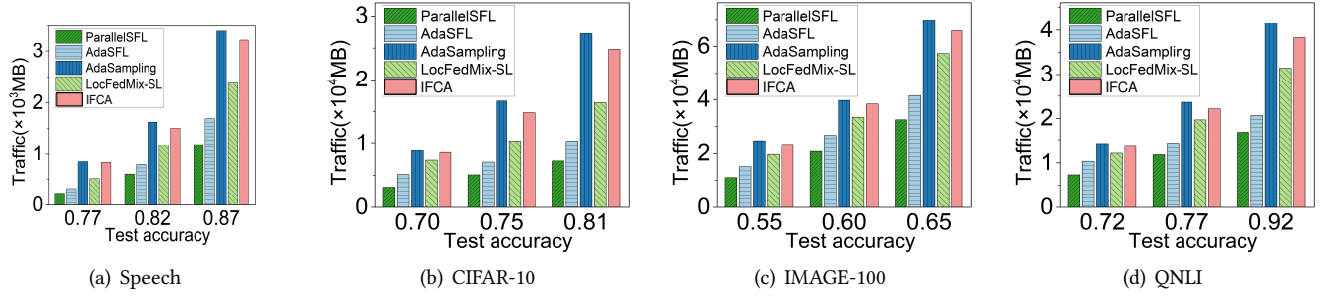


Figure 6: Network traffic consumption of five approaches when achieving different target accuracies.

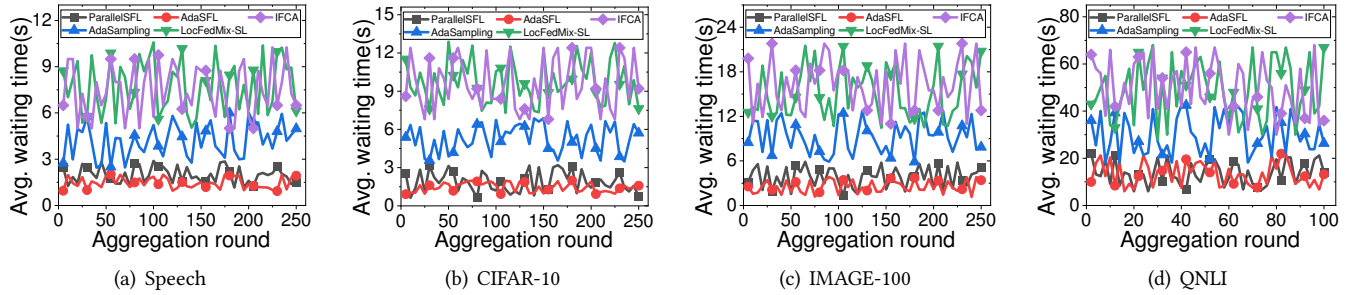


Figure 7: Average waiting time of five approaches on the four datasets.

the final accuracy by about 15.7%, 5.74%, 16.06% and 10.63% for VGG16 on IMAGE-100, compared to AdaSFL, AdaSampling, LocFedMix-SL, and IFCA, respectively. Moreover, as shown in Fig. 5(d), ParallelSFL achieves 90.11% accuracy in 11,445s for RoBERTa on QNLI, while AdaSFL, AdaSampling, LocFedMix-SL, and IFCA take 14,293s, 17,306s, 23,403s, and 30,724s to reach the accuracy of 84.51%, 87.33%, 84.62%, and 85.76%, respectively. These results demonstrate that ParallelSFL is effective in simultaneously tackling the system and statistical heterogeneity.

Thirdly, to illustrate the advantage of ParallelSFL in saving communication resource, we illustrate the network traffic consumption of these approaches when achieving different target accuracies in Fig. 6. By the results, the network traffic consumption of all approaches increases with the target accuracy for all the four datasets. Furthermore, ParallelSFL always consumes the least network traffic among all approaches. In addition, model splitting (*i.e.*, ParallelSFL, AdaSFL and LocFedMix-SL) helps to save much more network traffic compared to typical FL approaches (*i.e.*, IFCA and AdaSampling). AdaSFL with adaptive local updating frequency reduces the network traffic consumption while ParallelSFL with adaptive worker clustering further reduces the network traffic consumption. Specifically, As shown in Fig. 6(a), when achieving 87% accuracy, ParallelSFL, AdaSFL and LocFedMix-SL consume 1,192MB, 1,692MB and 2,398MB,

respectively, while AdaSampling and IFCA consume 3,403MB and 3,228MB for CNN on Speech. By Fig. 6(b), for training AlexNet on CIFAR-10 to achieve 81% accuracy, ParallelSFL reduces the network traffic consumption by about 3,012MB, 20,018MB, 9,116MB, and 17,564MB, compared to AdaSFL, AdaSampling, LocFedMix-SL, and IFCA, respectively. Besides, as illustrated in Fig. 6(c), ParallelSFL saves network traffic consumption by about 21%, 53%, 43%, and 49% when achieving 65% accuracy for VGG16 on IMAGE-100, compared to the baselines (*i.e.*, AdaSFL, AdaSampling, LocFedMix-SL, and IFCA). Moreover, by Fig. 6(d), for training RoBERTa on QNLI to achieve 92% accuracy, ParallelSFL reduces the network traffic consumption by about 3,783MB, 24,747MB, 14,537MB, and 21,422MB, compared to AdaSFL, AdaSampling, LocFedMix-SL, and IFCA, respectively. These results demonstrate that ParallelSFL is effective in saving network traffic consumption.

Finally, to further demonstrate the robustness of ParallelSFL towards system heterogeneity, we illustrate the average waiting time of the five approaches on both datasets in Fig. 7. AdaSFL with adaptive and diverse batch sizes for heterogeneous workers achieves the least waiting time, but the waiting time of ParallelSFL is close to AdaSFL and is much less than that of other approaches. For instance, as shown in Fig. 7(a), the average waiting time of ParallelSFL is 1.5s for CNN on Speech while AdaSFL, AdaSampling, LocFedMix-SL,

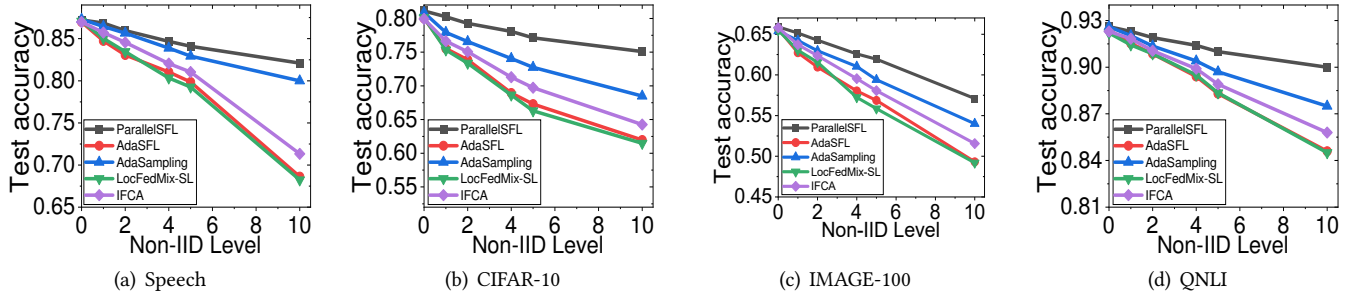


Figure 8: Test accuracy varies with different non-IID levels.

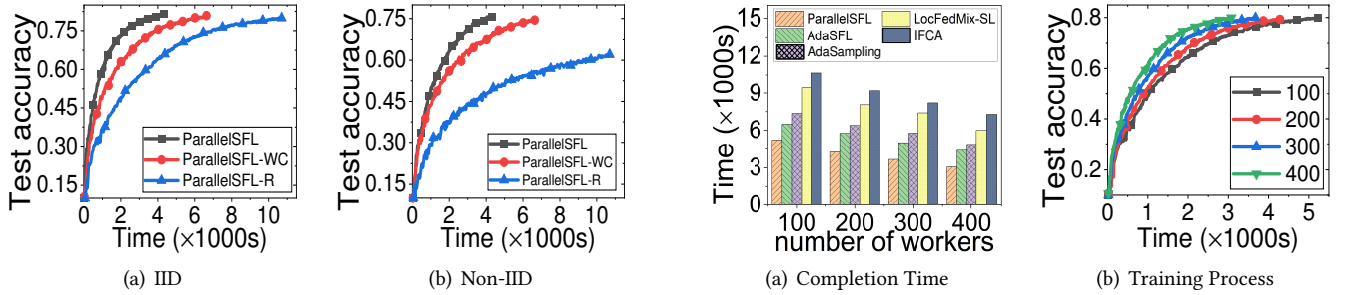


Figure 9: Effects of worker clustering and local updating frequency optimization.

and IFCA incur that of 1.3s, 4.4s, 7.3s and 7.1s, respectively. Similarly, by Fig. 7(b), compared to AdaSampling, LocFedMix-SL and IFCA, ParallelSFL reduces the average waiting time to train AlexNet on CIFAR-10 by about 53%, 67% and 62%, respectively. Considering the workers with varying capacities, LocFedMix-SL and IFCA do not consider system heterogeneity, thus leading to non-negligible waiting time. AdaSampling with adaptive worker sampling reduces the average waiting time to a certain extent. Specifically, by Fig. 7(c), ParallelSFL reduces the average waiting time for VGG16 on IMAGE-100 by about 63%, 80%, and 79%, compared to AdaSampling, LocFedMix-SL, and IFCA, respectively. Moreover, by Fig. 7(d), the average waiting time of ParallelSFL is 10.1s for RoBERTa on QNLI while AdaSampling, AdaSFL, LocFedMix-SL, and IFCA incur an average waiting time of 9.6s, 34.5s, 53.7s, and 51.3s, respectively. In general, these results illustrate that ParallelSFL overcomes the challenge of system heterogeneity well, compared to existing methods.

5.3 Effect of Non-IID Levels

To demonstrate the effectiveness of ParallelSFL in handling non-IID data, we present the test accuracy of different approaches at varying non-IID levels in Fig. 8, in which the model accuracy of the five approaches on all the datasets

Figure 10: Performance comparison with different number of workers.

decreases as the non-IID level increases. However, ParallelSFL consistently outperforms the other approaches on all datasets. AdaSFL and LocFedMix-SL, without considering the challenges of statistical heterogeneity, exhibit the lowest model accuracy on non-IID datasets. To tackle statistical heterogeneity, IFCA, which estimates the cluster identities, and AdaSampling, which designs an adaptive worker sampling algorithm, mitigate the impact of non-IID data on model training to some extent. Specifically, as illustrated in Fig. 8(a), ParallelSFL can achieve improvement of test accuracy by about 19.61%, 2.54%, 20.33%, and 15.13% on Speech with non-IID level of $p=10$, compared to the baselines (*i.e.*, AdaSFL, AdaSampling, LocFedMix-SL and IFCA). Notably, by Fig. 8(b), with non-IID level of $p=10$ on CIFAR-10, ParallelSFL achieves an improvement of final accuracy by about 21.17%, 9.65%, 22.15%, 16.84%, compared to the baselines (*i.e.*, AdaSFL, AdaSampling, LocFedMix-SL, IFCA). Besides, as shown in Fig. 8(c), while transitioning from IID to non-IID level of $p=10$ on IMAGE-100, ParallelSFL, AdaSampling and IFCA suffer from only 13.35%, 17.41% and 21.64% loss in accuracy, while the accuracy loss for AdaSFL, and LocFedMix-SL is 24.89% and 24.86%, respectively. Moreover, by Fig. 8(d), with non-IID level of $p=10$ on QNLI, ParallelSFL, AdaSampling and IFCA achieve 90.11%, 87.33%, and 85.76% accuracy, while

LocFedMix-SL and AdaSFL only achieve 84.51% and 84.62% accuracy. Collectively, these results further demonstrate the advantage of ParallelSFL with effective cluster clustering in addressing statistical heterogeneity.

5.4 Effect of Key Strategies

There are two key strategies of ParallelSFL, *i.e.*, worker clustering and local updating frequency optimization, being developed to enhance the performance of SFL. Herein, we conduct several sets of experiments for training AlexNet on CIFAR-10 with IID distribution ($p=0$) and non-IID distribution ($p=10$) to evaluate the effectiveness of the two critical strategies. We adopt the ParallelSFL without local updating frequency optimization (ParallelSFL-WC) and typical SFL with random worker clustering (ParallelSFL-R) as the baselines. Concretely, in ParallelSFL-WC, the PS assigns the identical and fixed local updating frequency for each cluster, while in ParallelSFL-R, the PS randomly partitions the workers into several clusters and determines the top worker for each cluster with identical and fixed local updating frequency. By Fig. 9, ParallelSFL-WC converges much faster than ParallelSFL-R on the IID dataset, and ParallelSFL-WC also achieves higher test accuracy than ParallelSFL-R on the non-IID dataset. Specifically, ParallelSFL-WC reduces the total training time by about 38% and improves the final test accuracy by about 20.06% on the non-IID dataset, compared to ParallelSFL-R. The results illustrate that our worker clustering strategy is effective in addressing the system and statistical heterogeneity. Besides, powered by the local updating frequency optimization among worker clusters, ParallelSFL speeds up training by about 1.56 $\times$ compared to ParallelSFL-WC on both IID and non-IID datasets. The results reflect the positive roles of worker clustering and local updating frequency optimization in ParallelSFL.

5.5 Effect of System Scales

In this section, to demonstrate the robustness of ParallelSFL, we evaluate the performance of ParallelSFL and baselines with different scales of participating workers. We conduct several sets of experiments for training AlexNet on CIFAR-10 with four scales (*e.g.*, 100, 200, 300, 400) through extensive simulation experiments, which are conducted on an AMAX deep learning workstation equipped with an Intel(R) Xeon(R) Platinum 8358P CPU @ 2.60GHz, 4 NVIDIA GeForce RTX A6000 GPUs (48GB GPU memory each) and 512 GB RAM. The results of completion time to achieve 80% accuracy for these approaches are presented in Fig. 10(a), while the training processes of different scales for ParallelSFL are presented in Fig. 10(b). As the number of participating workers increases, all approaches achieve faster convergence. The reason is that the number of samples on a worker is limited and

more workers contribute more local data for model training in each round, thus speeding up model training. For instance, ParallelSFL with 400 workers reduces the total training time by about 41%, 28%, 16%, compared to ParallelSFL with 100, 200, and 300 workers, respectively. In addition, ParallelSFL also achieves a speedup of 1.33 $\times$ ~2.27 $\times$ to reach the target accuracy, compared to the baselines (*i.e.*, AdaSFL, AdaSampling, LocFedMix-SL, IFCA) regarding the different scales of participating workers. These results further illustrate the robustness and advantage of ParallelSFL.

6 CONCLUSION

In this paper, we have proposed a novel SFL framework, named ParallelSFL, which integrates the advantages of FL and SFL and is designed to tackle heterogeneity issues by effective cluster partitioning. Under the heterogeneity restrictions, ParallelSFL defines a utility function to serve as a comprehensive metric for estimating waiting time and data distribution of worker clusters, and it contributes to balancing the trade-off between training efficiency and model accuracy. The experimental results show that the ParallelSFL can speed up the model training by at least 1.36 $\times$ and improve model accuracy by at least 5% in heterogeneous scenarios, compared to the baselines.

ACKNOWLEDGMENT

This article is supported in part by the National Key Research and Development Program of China (No. 2021YFB3301500); in part by the National Science Foundation of China (NSFC) under Grants 62102391, 61936015, and 62132019; in part by USTC Research Funds of the Double First-Class Initiative (No. WK2150110030).

REFERENCES

- [1] Biljana L Risteska Stojkoska and Kire V Trivodaliev. A review of internet of things for smart home: Challenges and solutions. *Journal of cleaner production*, 140:1454–1464, 2017.
- [2] Sharu Bansal and Dilip Kumar. Iot ecosystem: A survey on devices, gateways, operating systems, middleware and communication. *International Journal of Wireless Information Networks*, 27:340–364, 2020.
- [3] Diana Yacchirema, Jara Suárez de Puga, Carlos Palau, and Manuel Esteve. Fall detection system for elderly people using iot and big data. *Procedia computer science*, 130:603–610, 2018.
- [4] Sugandh Kumar Chaudhary, Syed Yousuff, NP Meghana, TS Ashwin, and Ram Mohana Reddy Guddeti. A multi-protocol home automation system using smart gateway. *Wireless Personal Communications*, 116:2367–2390, 2021.
- [5] Thilagamani Sathasivam, TA Janani, S Pavithra, and R Preethy. Iot-based smart baby cradle: A review. *ICT with Intelligent Applications: Proceedings of ICTIS 2022, Volume 1*, pages 589–603, 2022.
- [6] M Udin Harun Al Rasyid, Depandi Enda, and Ferry Astika Saputra. Smart home system for fire detection monitoring based on wireless sensor network. In *2019 International Electronics Symposium (IES)*, pages 189–194. IEEE, 2019.

- [7] Nikhil Komalapati, Varun Chowdary Yarra, Lakshmi Anantha Vyas Kancharla, and TN Shankar. Smart fire detection and surveillance system using iot. In *2021 International Conference on Artificial Intelligence and Smart Systems (ICAIS)*, pages 1386–1390. IEEE, 2021.
- [8] Mohamed Faisal Elrawy, Ali Ismail Awad, and Hesham FA Hamed. Intrusion detection systems for iot-based smart environments: a survey. *Journal of Cloud Computing*, 7(1):1–20, 2018.
- [9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [10] Riaan Rudman and Rikus Bruwer. Defining web 3.0: opportunities and challenges. *The electronic library*, 34(1):132–154, 2016.
- [11] Xiaofei Wang, Yiwen Han, Chenyang Wang, Qiyang Zhao, Xu Chen, and Min Chen. In-edge ai: Intelligentizing mobile edge computing, caching and communication by federated learning. *IEEE Network*, 33(5):156–165, 2019.
- [12] En Li, Liekang Zeng, Zhi Zhou, and Xu Chen. Edge ai: On-demand accelerating deep neural network inference via edge computing. *IEEE Transactions on Wireless Communications*, 19(1):447–457, 2019.
- [13] Andrew Hard, Kanishka Rao, Rajiv Mathews, Swaroop Ramaswamy, Françoise Beaufays, Sean Augenstein, Hubert Eichner, Chloé Kiddon, and Daniel Ramage. Federated learning for mobile keyboard prediction. *arXiv preprint arXiv:1811.03604*, 2018.
- [14] Peter Kairouz, H Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. Advances and open problems in federated learning. *arXiv preprint arXiv:1912.04977*, 2019.
- [15] Rui Han, Qinglong Zhang, Chi Harold Liu, Guoren Wang, Jian Tang, and Lydia Y Chen. Legodnn: block-grained scaling of deep neural networks for mobile vision. In *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*, pages 406–419, 2021.
- [16] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR, 2017.
- [17] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- [18] Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. Deep double descent: Where bigger models and more data hurt. *Journal of Statistical Mechanics: Theory and Experiment*, 2021(12):124003, 2021.
- [19] Chaoyue Niu, Fan Wu, Shaojie Tang, Lifeng Hua, Rongfei Jia, Chengfei Lv, Zhihua Wu, and Guihai Chen. Billion-scale federated learning on mobile clients: A submodel design with tunable privacy. In *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*, pages 1–14, 2020.
- [20] Wuyang Zhang, Zhezhi He, Luyang Liu, Zhenhua Jia, Yunxin Liu, Marco Gruteser, Dipankar Raychaudhuri, and Yanyong Zhang. Elf: accelerate high-resolution mobile deep vision with content-aware parallel offloading. In *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*, pages 201–214, 2021.
- [21] Jihong Park, Sumudu Samarakoon, Anis Elgabli, Joongheon Kim, Mehdi Bennis, Seong-Lyun Kim, and Mérouane Debbah. Communication-efficient and distributed learning over wireless networks: Principles and applications. *Proceedings of the IEEE*, 109(5):796–819, 2021.
- [22] Shraman Pal, Mansi Uniyal, Jihong Park, Praneeth Vepakomma, Ramesh Raskar, Mehdi Bennis, Moongu Jeon, and Jinho Choi. Server-side local gradient averaging and learning rate acceleration for scalable split learning. *arXiv preprint arXiv:2112.05929*, 2021.
- [23] Yunming Liao, Yang Xu, Hongli Xu, Lun Wang, Zhiwei Yao, and Chunming Qiao. Mergesfl: Split federated learning with feature merging and batch size regulation. *arXiv preprint arXiv:2311.13348*, 2023.
- [24] Seungeun Oh, Jihong Park, Praneeth Vepakomma, Sihun Baek, Ramesh Raskar, Mehdi Bennis, and Seong-Lyun Kim. Locfedmix-sl: Localize, federate, and mix for improved scalability, convergence, and latency in split learning. In *Proceedings of the ACM Web Conference 2022*, pages 3347–3357, 2022.
- [25] Chandra Thapa, Pathum Chamikara Mahawaga Arachchige, Seyit Camtepe, and Lichao Sun. Splitfed: When federated learning meets split learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 8485–8493, 2022.
- [26] Dong-Jun Han, Hasnain Irshad Bhatti, Jungmoon Lee, and Jaekyun Moon. Accelerating federated learning with split learning on locally generated losses. In *ICML 2021 Workshop on Federated Learning for User Privacy and Data Confidentiality*. ICML Board, 2021.
- [27] Ali Abedi and Shehroz S Khan. Fedsl: Federated split learning on distributed sequential data in recurrent neural networks. *arXiv preprint arXiv:2011.03180*, 2020.
- [28] Chen Zhao, Zhipeng Gao, Qian Wang, Zijia Mo, and Xinlei Yu. Fedgan: A federated semi-supervised learning from non-iid data. In *International Conference on Wireless Algorithms, Systems, and Applications*, pages 181–192. Springer, 2022.
- [29] Weiming Zhuang, Yonggang Wen, and Shuai Zhang. Divergence-aware federated self-supervised learning. *arXiv preprint arXiv:2204.04385*, 2022.
- [30] Yunming Liao, Yang Xu, Hongli Xu, Lun Wang, and Chen Qian. Adaptive configuration for heterogeneous participants in decentralized federated learning. In *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*, pages 1–10. IEEE, 2023.
- [31] Lun Wang, Yang Xu, Hongli Xu, Min Chen, and Liusheng Huang. Accelerating decentralized federated learning in heterogeneous edge computing. *IEEE Transactions on Mobile Computing*, 2022.
- [32] Yuexiang Xie, Zhen Wang, Dawei Gao, Daoyuan Chen, Liuyi Yao, Weirui Kuang, Yaliang Li, Bolin Ding, and Jingren Zhou. Federatedscope: A flexible federated learning platform for heterogeneity. *Proceedings of the VLDB Endowment*, 16(5):1059–1072, 2023.
- [33] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, Damon Civin, and Vikas Chandra. Federated learning with non-iid data. *arXiv preprint arXiv:1806.00582*, 2018.
- [34] Hao Wang, Zakhary Kaplan, Di Niu, and Baochun Li. Optimizing federated learning on non-iid data with reinforcement learning. In *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*, pages 1698–1707. IEEE, 2020.
- [35] Chenning Li, Xiao Zeng, Mi Zhang, and Zhichao Cao. Pyramidfl: A fine-grained client selection framework for efficient federated learning. In *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking*, pages 158–171, 2022.
- [36] Suo Chen, Yang Xu, Hongli Xu, Zhida Jiang, and Chunming Qiao. Decentralized federated learning with intermediate results in mobile edge computing. *IEEE Transactions on Mobile Computing*, 2022.
- [37] Fan Lai, Xiangfeng Zhu, Harsha V Madhyastha, and Mosharaf Chowdhury. Oort: Efficient federated learning via guided participant selection. In *OSDI*, pages 19–35, 2021.
- [38] Yang Xu, Yunming Liao, Hongli Xu, Zhenguo Ma, Lun Wang, and Jianchun Liu. Adaptive control of local updating and model compression for efficient federated learning. *IEEE Transactions on Mobile Computing*, 2022.
- [39] Wei Yang Bryan Lim, Nguyen Cong Luong, Dinh Thai Hoang, Yutao Jiao, Ying-Chang Liang, Qiang Yang, Dusit Niyato, and Chunyan Miao.

- Federated learning in mobile edge networks: A comprehensive survey. *IEEE Communications Surveys & Tutorials*, 22(3):2031–2063, 2020.
- [40] Jilin Zhang, Hangdi Tu, Yongjian Ren, Jian Wan, Li Zhou, Mingwei Li, and Jue Wang. An adaptive synchronous parallel strategy for distributed machine learning. *IEEE Access*, 6:19222–19230, 2018.
- [41] Naram Mhaisen, Alaa Awad Abdellatif, Amr Mohamed, Aiman Erbad, and Mohsen Guizani. Optimal user-edge assignment in hierarchical federated learning based on statistical properties and network topology constraints. *IEEE Transactions on Network Science and Engineering*, 9(1):55–66, 2021.
- [42] Otkrist Gupta and Ramesh Raskar. Distributed learning of deep neural network over multiple agents. *Journal of Network and Computer Applications*, 116:1–8, 2018.
- [43] Praneeth Vepakomma, Otkrist Gupta, Tristan Swedish, and Ramesh Raskar. Split learning for health: Distributed deep learning without sharing raw patient data. *arXiv preprint arXiv:1812.00564*, 2018.
- [44] Minsu Kim, Alexander DeRieux, and Walid Saad. A bargaining game for personalized, energy efficient split learning over wireless networks. In *2023 IEEE Wireless Communications and Networking Conference (WCNC)*, pages 1–6. IEEE, 2023.
- [45] Praveen Joshi, Chandra Thapa, Seyit Camtepe, Mohammed Hasanuz-zamana, Ted Scully, and Haithem Afli. Splitfed learning without client-side synchronization: Analyzing client-side split network portion size to overall performance. *arXiv preprint arXiv:2109.09246*, 2021.
- [46] Yunming Liao, Yang Xu, Hongli Xu, Zhiwei Yao, Lun Wang, and Chunming Qiao. Accelerating federated learning with data and model parallelism in edge computing. *IEEE/ACM Transactions on Networking*, 2023.
- [47] Yunming Liao, Yang Xu, Hongli Xu, Lun Wang, Chen Qian, and Chunming Qiao. Decentralized federated learning with adaptive configuration for heterogeneous participants. *IEEE Transactions on Mobile Computing*, 2023.
- [48] Bing Luo, Wenli Xiao, Shiqiang Wang, Jianwei Huang, and Leandros Tassioulas. Tackling system and statistical heterogeneity for federated learning with adaptive client sampling. In *IEEE INFOCOM 2022-IEEE conference on computer communications*, pages 1739–1748. IEEE, 2022.
- [49] Sarhad Arisdakessian, Omar Abdel Wahab, Azzam Mourad, and Hadi Otrouk. Towards instant clustering approach for federated learning client selection. In *2023 International Conference on Computing, Networking and Communications (ICNC)*, pages 409–413. IEEE, 2023.
- [50] Enmao Diao, Jie Ding, and Vahid Tarokh. Heterofl: Computation and communication efficient federated learning for heterogeneous clients. In *International Conference on Learning Representations*, 2020.
- [51] Felix Sattler, Klaus-Robert Müller, and Wojciech Samek. Clustered federated learning: Model-agnostic distributed multitask optimization under privacy constraints. *IEEE transactions on neural networks and learning systems*, 32(8):3710–3722, 2020.
- [52] Jaemin Shin, Yuanchun Li, Yunxin Liu, and Sung-Ju Lee. Fedbalancer: data and pace control for efficient federated learning on heterogeneous clients. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services*, pages 436–449, 2022.
- [53] Avishek Ghosh, Jichan Chung, Dong Yin, and Kannan Ramchandran. An efficient framework for clustered federated learning. *Advances in Neural Information Processing Systems*, 33:19586–19597, 2020.
- [54] Saeed Vahidian, Mahdi Morafah, Weijia Wang, Vyacheslav Kungurtsev, Chen Chen, Mubarak Shah, and Bill Lin. Efficient distribution similarity identification in clustered federated learning via principal angles between client data subspaces. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 10043–10052, 2023.
- [55] Avishek Ghosh, Jichan Chung, Dong Yin, and Kannan Ramchandran. An efficient framework for clustered federated learning. *IEEE Transactions on Information Theory*, 68(12):8076–8091, 2022.
- [56] Shiqiang Wang, Tiffany Tuor, Theodoros Salonidis, Kin K Leung, Christian Makaya, Ting He, and Kevin Chan. When edge meets learning: Adaptive control for resource-constrained distributed machine learning. In *IEEE INFOCOM 2018-IEEE Conference on Computer Communications*, pages 63–71. IEEE, 2018.
- [57] Zhenguo Ma, Yang Xu, Hongli Xu, Zeyu Meng, Liusheng Huang, and Yinxing Xue. Adaptive batch size for federated learning in resource-constrained edge computing. *IEEE Transactions on Mobile Computing*, 2021.
- [58] David Leroy, Alice Coucke, Thibaut Lavril, Thibault Gisselbrecht, and Joseph Dureau. Federated learning for keyword spotting. In *ICASSP 2019-2019 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pages 6341–6345. IEEE, 2019.
- [59] Xinchun Lyu, Chenshan Ren, Wei Ni, Hui Tian, Ren Ping Liu, and Y Jay Guo. Multi-timescale decentralized online orchestration of software-defined networks. *IEEE Journal on Selected Areas in Communications*, 36(12):2716–2730, 2018.
- [60] Xinchun Lyu, Chenshan Ren, Wei Ni, Hui Tian, Ren Ping Liu, and Eryk Dutkiewicz. Optimal online data partitioning for geo-distributed machine learning in edge of wireless networks. *IEEE Journal on Selected Areas in Communications*, 37(10):2393–2406, 2019.
- [61] John R Hershey and Peder A Olsen. Approximating the kullback leibler divergence between gaussian mixture models. In *2007 IEEE International Conference on Acoustics, Speech and Signal Processing-ICASSP'07*, volume 4, pages IV–317. IEEE, 2007.
- [62] Goldberger, Gordon, and Greenspan. An efficient image similarity measure based on approximations of kl-divergence between two gaussian mixtures. In *Proceedings Ninth IEEE International conference on computer vision*, pages 487–493. IEEE, 2003.
- [63] Shenghui Li, Edith Ngai, Fanghua Ye, and Thiemo Voigt. Auto-weighted robust federated learning with corrupted data sources. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 13(5):1–20, 2022.
- [64] Zexi Li, Tao Lin, Xinyi Shang, and Chao Wu. Revisiting weighted aggregation in federated learning with neural networks. *arXiv preprint arXiv:2302.10911*, 2023.
- [65] Chandra Thapa, Mahawaga Arachchige Pathum Chamikara, and Seyit A Camtepe. Advancements of federated learning towards privacy preservation: from federated learning to split learning. *Federated Learning Systems: Towards Next-Generation AI*, pages 79–109, 2021.
- [66] Maoqiang Wu, Guoliang Cheng, Dongdong Ye, Jiawen Kang, Rong Yu, Yuan Wu, and Miao Pan. Federated split learning with data and label privacy preservation in vehicular networks. *IEEE Transactions on Vehicular Technology*, 2023.
- [67] Badih Ghazi, Noah Golowich, Ravi Kumar, Pasin Manurangsi, and Chiyuan Zhang. Deep learning with label differential privacy. *Advances in neural information processing systems*, 34:27131–27145, 2021.
- [68] Ziyuan Yang, Yingyu Chen, Huijie Huangfu, Maosong Ran, Hui Wang, Xiaoxiao Li, and Yi Zhang. Dynamic corrected split federated learning with homomorphic encryption for u-shaped medical image networks. *IEEE Journal of Biomedical and Health Informatics*, 2023.
- [69] Dirk Merkel et al. Docker: lightweight linux containers for consistent development and deployment. *Linux j*, 239(2):2, 2014.
- [70] Nitin Naik. Building a virtual system of systems using docker swarm in multiple clouds. In *2016 IEEE International Symposium on Systems Engineering (ISSE)*, pages 1–3. IEEE, 2016.
- [71] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems*, pages 8026–8037, 2019.

- [72] Edgar Gabriel, Graham E Fagg, George Bosilca, Thara Angskun, Jack J Dongarra, Jeffrey M Squyres, Vishal Sahay, Prabhanjan Kambadur, Brian Barrett, Andrew Lumsdaine, et al. Open mpi: Goals, concept, and design of a next generation mpi implementation. In *European Parallel Virtual Machine/Message Passing Interface Users' Group Meeting*, pages 97–104. Springer, 2004.
- [73] Ajay Tirumala. Iperf: The tcp/udp bandwidth measurement tool. <http://dast.nlanr.net/Projects/Iperf/>, 1999.
- [74] Pete Warden. Speech commands: A dataset for limited-vocabulary speech recognition. *arXiv preprint arXiv:1804.03209*, 2018.
- [75] Alex Krizhevsky and Geoff Hinton. Convolutional deep belief networks on cifar-10. *Unpublished manuscript*, 40(7):1–9, 2010.
- [76] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25:1097–1105, 2012.
- [77] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115(3):211–252, 2015.
- [78] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.
- [79] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461*, 2018.
- [80] Tzu-Ming Harry Hsu, Hang Qi, and Matthew Brown. Measuring the effects of non-identical data distribution for federated visual classification. *arXiv preprint arXiv:1909.06335*, 2019.
- [81] Mikhail Yurochkin, Mayank Agarwal, Soumya Ghosh, Kristjan Greenewald, Nghia Hoang, and Yasaman Khazaeni. Bayesian nonparametric federated learning of neural networks. In *International conference on machine learning*, pages 7252–7261. PMLR, 2019.
- [82] Daniel Rothchild, Ashwinee Panda, Enayat Ullah, Nikita Ivkin, Ion Stoica, Vladimir Braverman, Joseph Gonzalez, and Raman Arora. Fetchsgd: Communication-efficient federated learning with sketching. In *International Conference on Machine Learning*, pages 8253–8265. PMLR, 2020.